

Break a model, then fix it

A three hour hands-on session on quantisation, and on proving the thing you quantised still works. This is the delivery plan: how the room spends the time, what runs on a laptop, and what I do when a third of it does not.

The arc, in four moves

1

Build it

Participants write the quantiser themselves. The grid, rounding, and a bounded straight through estimator. About forty lines, with a unit test that has to go green before anyone moves on.

2

Break it, on purpose

Three deliberate failures, one at a time, each a one line change. All three are bugs I actually shipped, and each one fails in a way that does not look like what it is.

3

Diagnose from the weights

Not from generated text. Participants learn to read the weight distribution and predict which break they are looking at before the answer is revealed.

4

Build the evaluation

The last hour builds the harness that catches the failure accuracy alone misses, and runs it against all four models from the session: one good, three broken.

The run sheet

0:00	15 min	Setup check and the premise. Everyone runs verify.py and sees PASS before anything starts.
0:15	25 min	Build the quantiser. One provided test must go green. Nobody proceeds on a red test.
0:40	20 min	Quantise a real checkpoint to eight bits. Generate text before and after. It still works.
1:00	10 min	Break.
1:10	40 min	Break it three ways, roughly twelve minutes each. Every break is a one line change.
1:50	30 min	Diagnose from the weight distribution. Predict the break, then reveal.
2:20	40 min	Build the evaluation. Run it on all four models. Watch accuracy alone pass a broken one.

The three things we break on purpose

The dead layer

Quantise a freshly initialised wide layer at three bits. Every weight rounds to zero. The network is an exactly zero function and no gradient can revive it.

Looks like: a model that refuses to train.

Actually is: the scale sitting in the wrong place, not a precision limit.

The outlier clip

Quantise the two projection matrices that no normalisation layer feeds, without giving them a scale. Validation loss goes from 2.53 to 7.73.

Looks like: not enough bits.

Actually is: a few hundred clipped weights, at sixteen bits, where the grid step is three in a hundred thousand.

The silent one

Quantise too far, then score it the obvious way. The model stops producing output and the benchmark rewards it for being cautious.

Looks like: a mediocre result.

Actually is: a dead model, and a metric that cannot tell the difference.

Three numbers, three diagnoses

100%

of weights in a wide layer round to zero at three bits. Count the zeros and the dead layer announces itself in one line.

7.47

the largest weight in a checkpoint everyone assumes is bounded by one. Take a per channel max and the outlier clip is obvious.

1.2%

of prompts a broken model still answered, while scoring a perfect 100 on one benchmark category. Count attempts, not just successes.

Every diagnosis in this workshop is a histogram or a counter.

Nobody needs to interpret generated text, which is the part that normally goes wrong in a room with mixed experience. The signal is a number that is obviously right or obviously not.

What it takes to run on a laptop

What participants need

Python 3.10 or newer, PyTorch, NumPy, Matplotlib. A setup script pins all of it. Total install is under five minutes on a normal connection.

What they do not need

No GPU. No cloud account. No vendor licence. No dataset download. The whole session is under ten minutes of actual compute, all of it on CPU.

The model

One small open weight checkpoint, around 150 MB, pinned by revision and cached by the setup script. Small enough to quantise in seconds, real enough that the failures are the real failures.

The fallback

An identical Colab notebook, cell for cell. Anyone whose local Python fights them moves there in under a minute and loses nothing.

Keeping thirty people in the same place

1

A checkpoint after every stage

The repository carries the finished state of each stage as its own branch. Anyone who falls behind checks out the next one and rejoins the room immediately, with working code they can read.

2

Tests, not my judgement

Each stage ends on a provided test. Green means move on. That replaces me walking round asking thirty people whether they are done.

3

Pairs, by choice

I ask people to sit in twos at the start. It halves the number of broken environments and it means the faster half of the room is occupied teaching rather than waiting.

4

One helper past twenty

Below twenty I can run the floor alone. Above that I would ask for one volunteer who has done the setup in advance, purely for environment triage in the first fifteen minutes.

What goes wrong, and what I do

Venue wifi is slow or absent

The setup script runs before arrival and caches everything. I bring the environment and the checkpoint on USB sticks as well.

Someone's Python is unfixable

Colab notebook, identical cells. Decided in a minute, instead of debugged for half an hour.

The room runs at two different speeds

Stage branches. The fast half goes on to an optional fourth break; nobody waits and nobody is stranded.

I lose time early and the clock slips

The evaluation hour is the designed overflow. I demo it instead of building it and the session still lands.

What people leave with

A working quantiser

Forty lines they wrote, that they understand, and that runs on any checkpoint they have access to on Monday morning.

A diagnosis routine

Three checks against a checkpoint: dead weight fraction, per channel maximum against the representable bound, and a liveness counter.

An evaluation pattern

A capability metric with a liveness metric beside it, which generalises well past quantisation to anything that can fail by going quiet.

The test is whether they can answer one question about their own model.

Will this checkpoint survive being quantised, and how would I know if it had not? If they can answer that without running the quantiser, the three hours worked.

What I would ask for

1

Three hours, and a cap of thirty

Two hours works with the evaluation demoed rather than built. Past thirty I would want one volunteer for setup triage.

2

The setup script in the joining email

A week ahead. This single thing is the difference between starting at 0:00 and starting at 0:25.

3

The January to March quarter, not November

November sits between four other commitments. Q1 also puts it after the silicon tapes out, so the hardware section is measured rather than projected.

Happy to run it in November if that is what the schedule needs. The answer above is about doing it well, not about whether I will do it.